OSM Data Library

Travis Guenther – tguenth@pdx.edu

I would like to opt out of the project theme. This is a project I've been wanting to work on for quite a while.

OpenStreetMap (OSM) is a database comprised of user-contributed geodata. The OpenStreetMap Foundation doesn't officially produce any software other than OSM's main website, and generally few resources are invested by businesses that use the data. Tools for editing, processing and rendering the data mostly originate from the community of open-source software that has grown around the organization.

Many of the apps for editing OSM data are only designed for making minor changes, e.g. adding an address or website to a business. More advanced, general-purpose editors are harder to develop, so there are only a few. The most popular general-purpose editor − used by about two thirds of contributors − is iD, a web app written in JavaScript. It has a relatively user-friendly interface, however its performance is not ideal. The second most popular is JOSM − a powerful Java app with decent performance, a complex and awkward interface, and about 20 years of tech-debt.

A new, modern editor would greatly benefit the community. However, this would be a very large project, partly because every editor implements its own methods of handling OSM data. There are many libraries for standard geodata types (i.e. OGC Simple Features), but these do not correspond exactly to OSM data types. Converting between OSM and OGC types is often necessary. Existing functions that do so are difficult to extract from their respective apps and are typically written in Java or JavaScript, which limit their application.

As a first step toward this goal, I intend to create a library for handling OSM data. This can be used by any projects that edit or render OSM data to substantially reduce the effort required to create them. For the scope of this class, the project goals are:

Primary goals

- Define OSM data types
- Convert GeoJson → OSM
- Convert OSM → GeoJson

Secondary goals

- Download data
- Upload data

Dependencies

- geo-types provides OGC data types
- geojson provides read/write functionality for GeoJson file format
- xml provides read/write functionality for XML format for OSM API

Definitions

- Coordinate pair − Geographic coordinates comprised of latitude and longitude values

OpenStreetMap terms:

- Tags − A map of key & value strings that describe an OSM feature
- Node − OSM feature type; coordinate pair; has tags
- Way − OSM feature type; a list of 2+ nodes; has tags
- Relation − OSM feature type; a list of other features paired with a “role” string; has tags including a “type” key
- Area − OSM pseudo-type; a Way that forms a ring (3+ unique nodes, begins and ends with the same node) and has tags with “area semantics” (vaguely defined)

GeoJson terms:

- GeoJson − A JSON-based file format for geographic features that uses the subset of 7 OGC geometry types listed below, each paired with a set of properties
- Properties − A flat JSON Object (map of key & value strings) that describe a feature
- Point − Geometry type; single coordinate pair
- LineString − Geometry type; ordered list of connected coordinate pairs
- Polygon − Geometry type; ordered list of connected coordinate pairs that form a ring and an optional list of non-intersecting inner “cut-out” rings
- MultiPoint − Geometry type; list of Point geometries
- MultiLineString − Geometry type; list of LineString geometries
- MultiPolygon − Geometry type; list of Polygon geometries
- GeometryCollection − Geometry type; list of any geometries

Goal: Convert OSM → GeoJson

- Node → if tags or no parent Way / Relation then Point
- Way → if is Area then Polygon else LineString
- Relation → if tag[type] == multipolygon then (if single outer ring then Polygon else MultiPolygon) else GeometryCollection

Goal: Convert GeoJson → OSM

Stage 1: Expand collections

- GeometryCollection → Relation & list<any>

Stage 2: Expand multi-geometry

- MultiPoint → Relation & list<Point>
- MultiLineString → Relation & list<LineString>
- MultiPolygon → Relation(tag[type]=multipolygon) & list<Polygon>

Stage 3: Convert basic types

- Point → Node
- LineString → Way & list<Node>
- Polygon → if no inner rings then Way else Relation(tag[type]=multipolygon)

Stage 4: Deduplication

- Untagged Nodes in Ways / Relations
- Untagged Ways in Relations

Goal: Download data

- Send query to OSM API
- Parse XML response

Goal: Upload data

- OAuth
- Create changeset
- Post to OSM API

Links

- OSM Read-only API
- OSM Editing API
- OSM XML format
- OsmChange XML format
- GeoRust